1.

The matrix_task is performing very computationally intensive operations:

- It executes a **10x10 matrix multiplication**.
- It utilizes 4 nested loops (i, j, k, l).
- It includes a simulation delay of **1 billion iterations**.
- Consequently, it occupies the CPU for an extended period.

2.

In FreeRTOS, the highest priority task always executes first.

- Initially, matrix_task is **Priority 3** (High) and communication_task is **Priority 2** (Low).
- When the Matrix task hogs the CPU, the Communication task is forced to wait (starvation).
- Although the Communication task is scheduled to run every 200ms, it cannot get CPU time until the matrix processing finishes.
- When the priority of the Communication task is increased (to **Priority 4**), it is able to run before the Matrix task.

3.

The completion time of matrix_task increases.

Reason:

- When the Communication task is set to Priority 4, it **preempts** the matrix_task.
- The Communication task interrupts the Matrix task while it is running.
- The Matrix task pauses, waits for the Communication task to finish, and then resumes.
- Due to these interruptions and context switches, the total execution time of the matrix task is prolonged.

4.

Period = Matrix Execution Time + 100ms

To measure the exact duration, you can use vApplicationTickHook():

1. Record the start tick when the Matrix task begins.

2. Record the end tick when it finishes.

3. The difference is the Period (convert ticks to seconds).

Estimation: Considering the simulation delay plus the matrix multiplication, it will likely take **several seconds** (approximately **3–5 seconds**, depending on the system clock speed).